

프로그래밍 파이썬



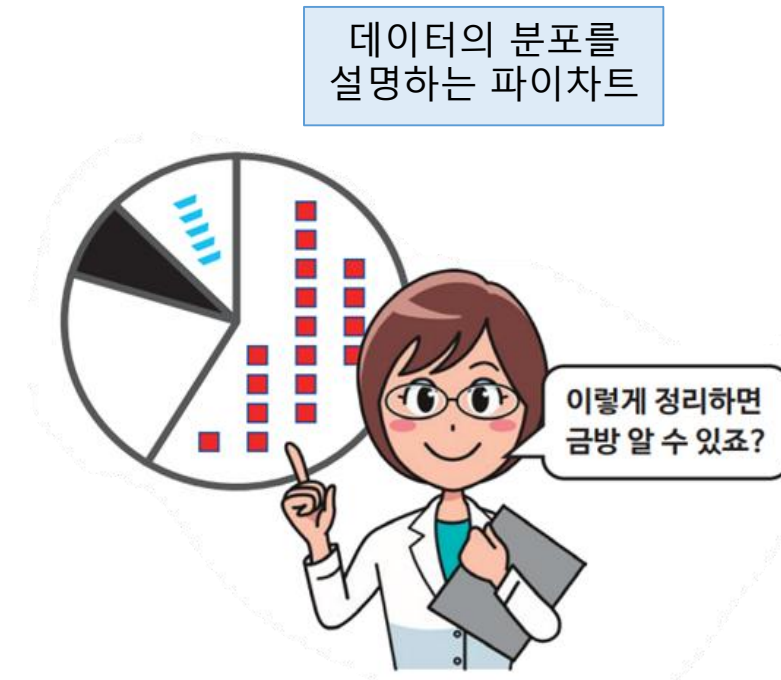
12장 넘파이(NumPy) MatPlot

학습목표

- 파이썬에는 풍부하고도 유용한 외부 라이브러리가 있음을 이해한다.
- 과학 계산에 필수적인 넘파이 라이브러리의 용도와 사용법에 대해 이해한다.
- 넘파이의 핵심적인 요소인 ndarray를 사용해 본다.
- ndarray와 리스트의 차이를 이해하고, 그 특성에 맞는 활용을 할 수 있다.
- 넘파이가 과학기술 분야와 머신 러닝 분야에 적합한 이유를 설명할 수 있다.
- ndarray와 선형대수 문제의 연관을 이해한다.
- 넘파이를 이용하여 다양한 행렬, 벡터 문제를 해결할 수 있다.
- 넘파이의 linalg 패키지를 이용하여 다양한 선형대수 문제를 풀 수 있다.
- 파이썬의 리스트와 넘파이의 ndarray를 연동하여 계산할 수 있는 기초 지식을 습득한다.

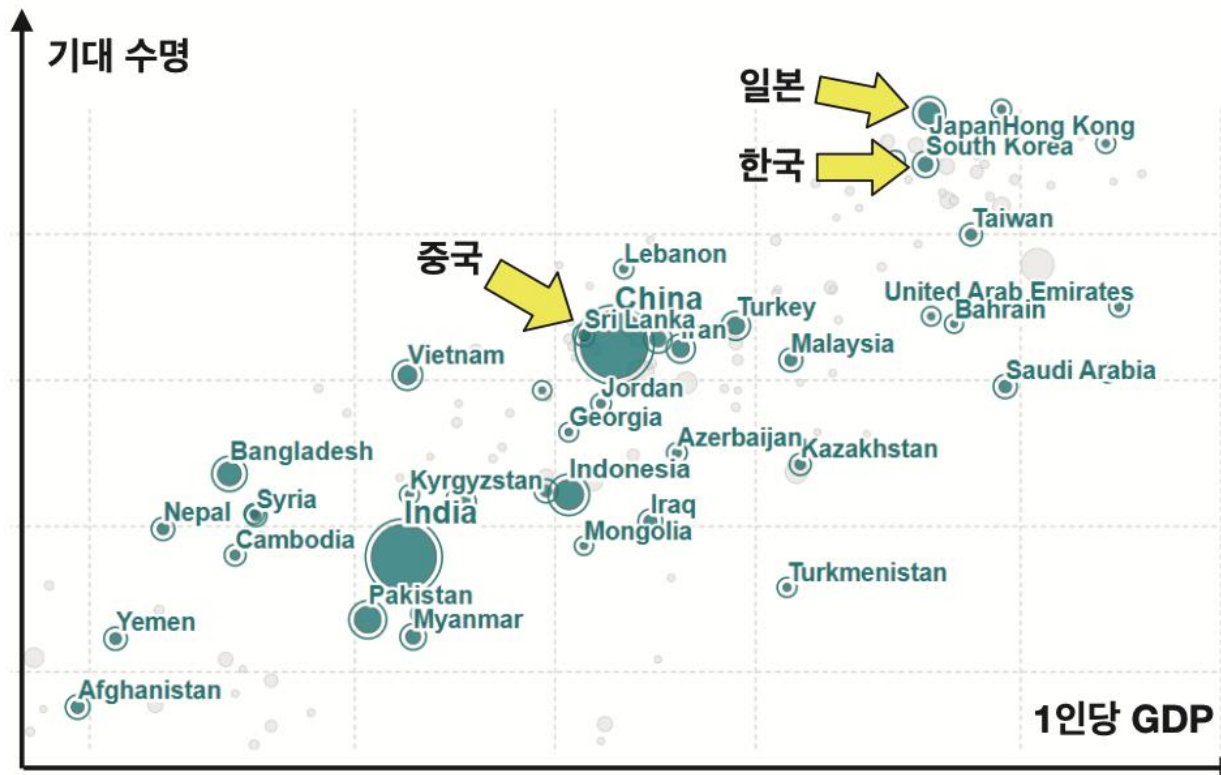
데이터 시각화(data visualization)

- **데이터 시각화**는 점이나 선, 막대 그래프등의 시각적 이미지를 사용하여 데이터를 화면에 표시하는 기술
- 사람들은 시각적으로 보이는 데이터를 더 직관적으로 이해함



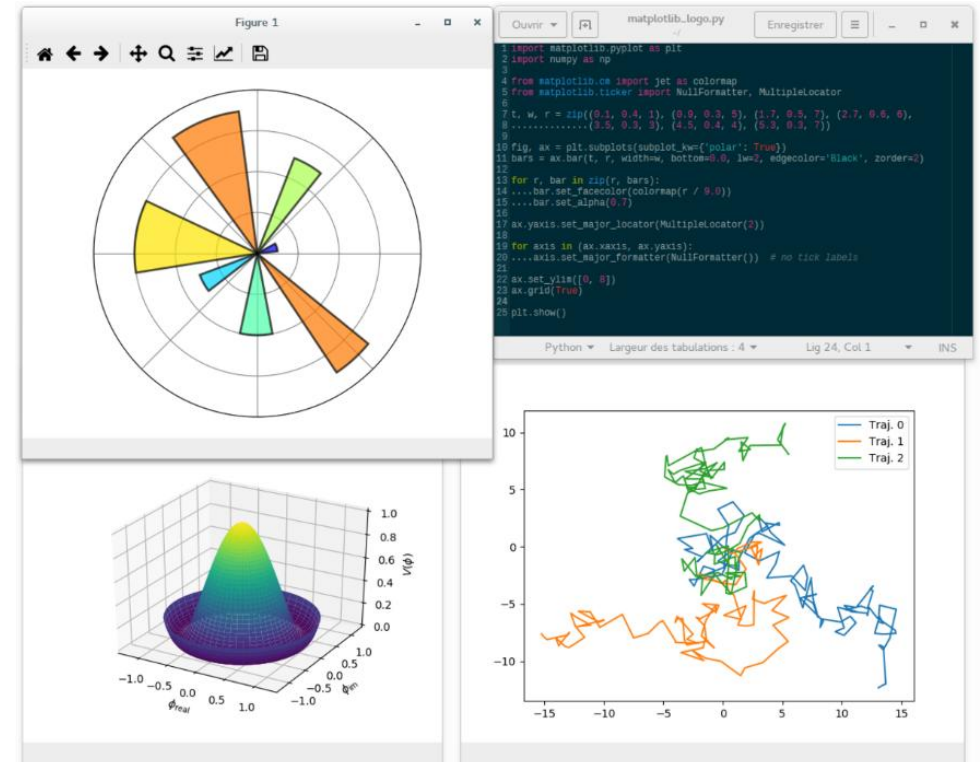
데이터 시각화로 가치있는 정보를 만든 사례

- 스웨덴의 한스 로슬링 Hans Rosling 교수가 만든 차트
- GDP(국내총생산)와 기대 수명 사이의 상관 관계

출처: Our World in Data, <https://ourworldindata.org/>

MatPlot

- MatPlot은 그래프를 그리는 라이브러리
- 가장 널리 사용되는 시각화 도구
 - 막대 그래프, 선 그래프, 산포도 그리는 용도
- MatPlot은 무료이고 오픈 소스이다



matplotlib 설치

```
C:\Users\WIT>pip install matplotlib
```

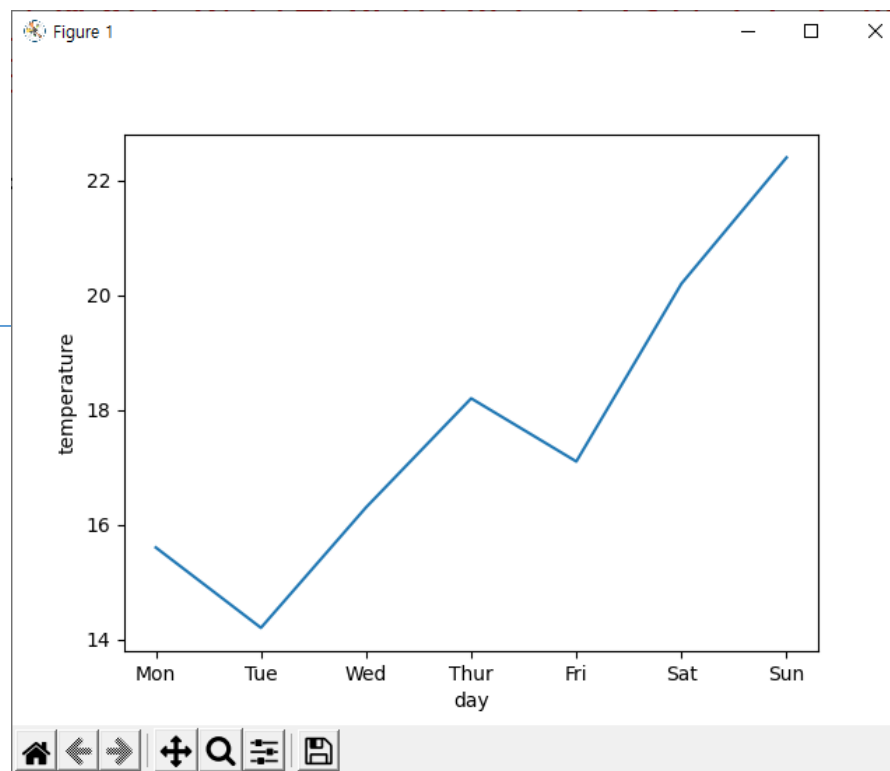
```
C:\Users\WIT>pip install matplotlib
Collecting matplotlib
  Downloading matplotlib-3.5.2-cp310-cp310-win_amd64.whl (7.2 MB)
    |████████████████████| 7.2 MB 6.8 MB/s
Collecting pillow>=6.2.0
  Downloading Pillow-9.1.1-cp310-cp310-win_amd64.whl (3.3 MB)
    |████████████████████| 3.3 MB 6.8 MB/s
Collecting pyparsing>=2.2.1
  Downloading pyparsing-3.0.9-py3-none-any.whl (98 kB)
    |████████████████████| 98 kB 6.4 MB/s
Collecting packaging>=20.0
  Downloading packaging-21.3-py3-none-any.whl (40 kB)
    |████████████████████| 40 kB 2.5 MB/s
Collecting fonttools>=4.22.0
  Downloading fonttools-4.33.3-py3-none-any.whl (930 kB)
    |████████████████████| 930 kB ...
Collecting cycler>=0.10
  Downloading cycler-0.11.0-py3-none-any.whl (6.4 kB)
Collecting kiwisolver>=1.0.1
  Downloading kiwisolver-1.4.2-cp310-cp310-win_amd64.whl (55 kB)
    |████████████████████| 55 kB 707 kB/s
Collecting python-dateutil>=2.7
  Downloading python_dateutil-2.8.2-py2.py3-none-any.whl (247 kB)
    |████████████████████| 247 kB 6.4 MB/s
Collecting numpy>=1.17
  Downloading numpy-1.22.4-cp310-cp310-win_amd64.whl (14.7 MB)
    |████████████████████| 14.7 MB 6.8 MB/s
Collecting six>=1.5
  Downloading six-1.16.0-py2.py3-none-any.whl (11 kB)
Installing collected packages: six, pyparsing, python-dateutil, pillow, packaging, numpy, kiwisolver, fonttools, cycler, matplotlib
Successfully installed cycler-0.11.0 fonttools-4.33.3 kiwisolver-1.4.2 matplotlib-3.5.2 numpy-1.22.4 packaging-21.3 pillow-9.1.1 pyparsing-3.0.9 python-dateutil-2.8.2 six-1.16.0
WARNING: You are using pip version 21.2.4; however, version 22.1.2 is available.
You should consider upgrading via the 'C:\Users\WIT\AppData\Local\Programs\Python\Python310\python.exe
```

직선(꺾은선) 그래프

```
import matplotlib.pyplot as plt
```

```
X = [ "Mon", "Tue", "Wed", "Thur", "Fri", "Sat", "Sun" ]  
Y = [ 15.6, 14.2, 16.3, 18.2, 17.1, 20.2, 22.4 ]
```

```
plt.plot(X, Y)  
plt.xlabel("day")  
plt.ylabel("temperature")  
plt.show()
```



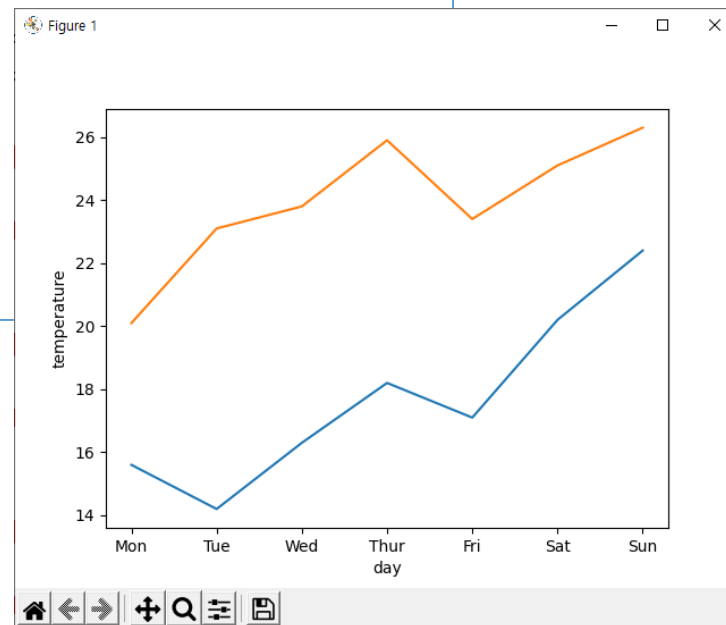
직선 그래프-값을 겹쳐서 표현

- 2개의 값을 겹쳐서 표현 - 최저 온도와 최고 온도

```
import matplotlib.pyplot as plt

X = [ "Mon", "Tue", "Wed", "Thur", "Fri", "Sat", "Sun" ]
Y1 = [ 15.6, 14.2, 16.3, 18.2, 17.1, 20.2, 22.4 ]
Y2 = [ 20.1, 23.1, 23.8, 25.9, 23.4, 25.1, 26.3 ]

plt.plot(X, Y1, X, Y2) #2개의 쌍을 표시
plt.xlabel("day")
plt.ylabel("temperature")
plt.show()
```



직선 그래프

- 그래프 타이틀과 레이블 표시

```
import matplotlib.pyplot as plt
```

```
X = [ "Mon", "Tue", "Wed", "Thur", "Fri", "Sat", "Sun" ]
```

```
Y1 = [15.6, 14.2, 16.3, 18.2, 17.1, 20.2, 22.4]
```

```
Y2 = [20.1, 23.1, 23.8, 25.9, 23.4, 25.1, 26.3]
```

```
plt.plot(X, Y1, label="Seoul") # 분리시켜서 그려도 됨
```

```
plt.plot(X, Y2, label="Daegu") # 분리시켜서 그려도 됨
```

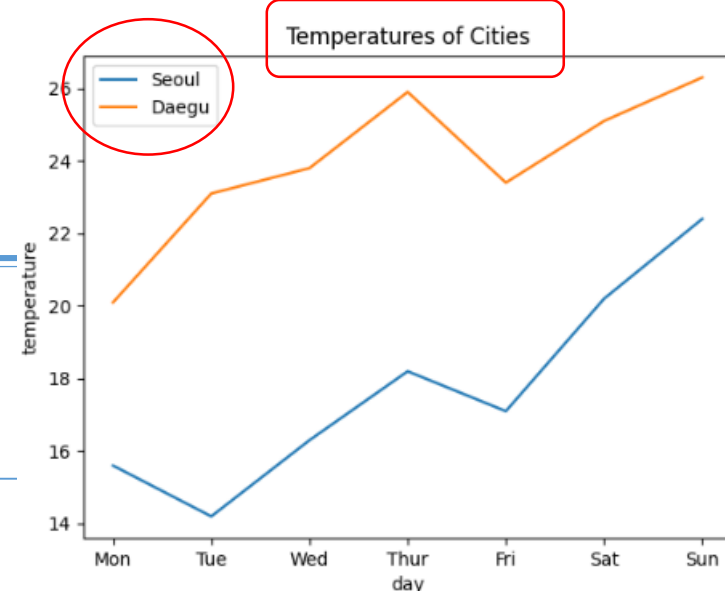
```
plt.xlabel("day")
```

```
plt.ylabel("temperature")
```

```
plt.legend(loc="upper left") # y축 값이 무엇인지 표시
```

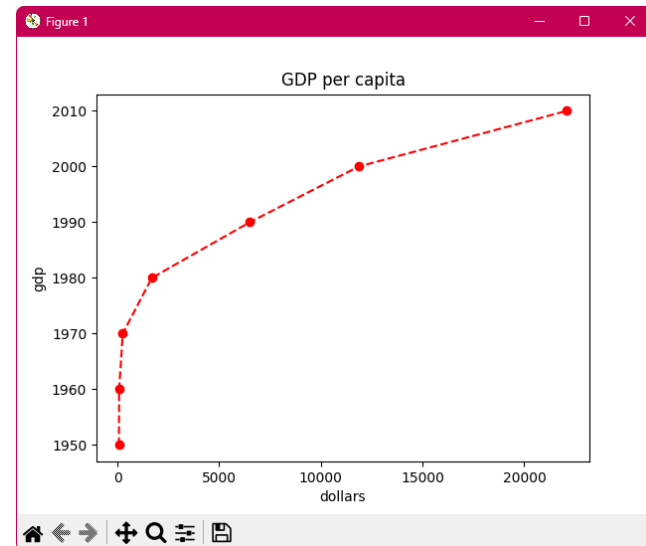
```
plt.title("Temperatures of Cities") #그래프 제목 표시
```

```
plt.show()
```

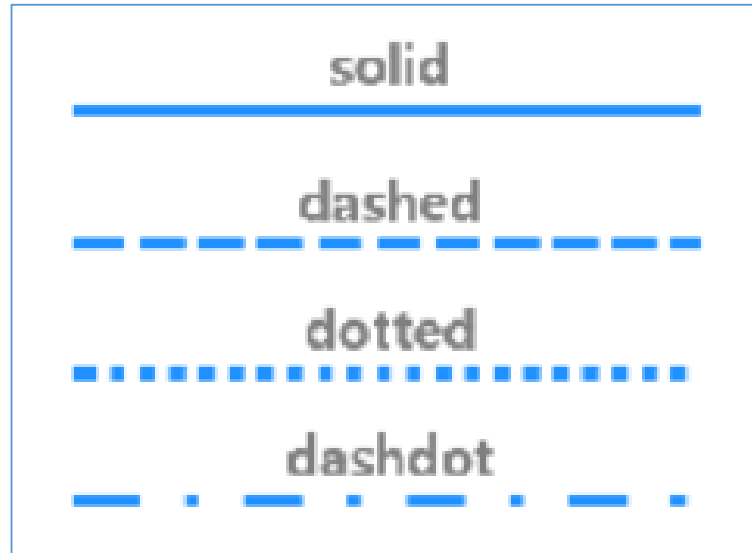


직선 그래프 색, marker, 선모양 지정

```
1 import matplotlib.pyplot as plt
2
3 # 우리나라의 연간 1인당 국민소득을 각각 years, gdp에 저장
4 years = [1950, 1960, 1970, 1980, 1990, 2000, 2010]
5 gdp = [67.0, 80.0, 257.0, 1686.0, 6505, 11865.3, 22105.3]
6
7
8 plt.title("GDP per capita") # 1인당 국민소득
9
10 plt.xlabel("dollars")
11 plt.ylabel("gdp")
12
13 plt.plot(gdp, years, color='red', marker='o', linestyle='dashed')
14 # 색, marker, 선모양 지정
15 plt.savefig("gdp_per_capita.png", dpi=600)
16 # png 이미지로 저장 가능
17
18 plt.show()
```



선모양 지정



 Solid

`plt.plot(x, y, '-')`

 Dashed

`plt.plot(x, y, '--')`

 Dotted

`plt.plot(x, y, ':')`

 Dash-dot

`plt.plot(x, y, '-.')`

다양한 색, 선 종류, 마커

Colors

character	color
'b'	blue
'g'	green
'r'	red
'c'	cyan
'm'	magenta
'y'	yellow
'k'	black
'w'	white

Line Styles

character	description
'-'	solid line style
'--'	dashed line style
'-.'	dash-dot line style
':'	dotted line style

Markers

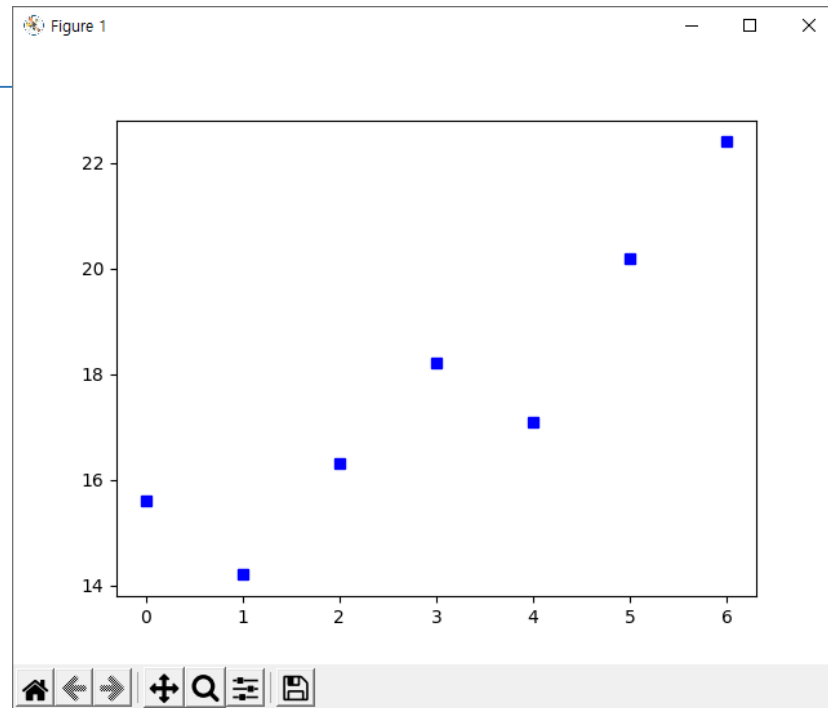
character	description
'.'	point marker
','	pixel marker
'o'	circle marker
'v'	triangle_down marker
'^'	triangle_up marker
'<'	triangle_left marker
'>'	triangle_right marker
'1'	tri_down marker
'2'	tri_up marker
'3'	tri_left marker
'4'	tri_right marker
's'	square marker
'p'	pentagon marker
'*'	star marker
'h'	hexagon1 marker
'H'	hexagon2 marker
'+'	plus marker
'x'	x marker
'D'	diamond marker
'd'	thin_diamond marker
' '	vline marker
'_'	hline marker

점선 그래프

- sb에서 s는 모양(square marker), b는 색(blue)을 의미

```
import matplotlib.pyplot as plt
```

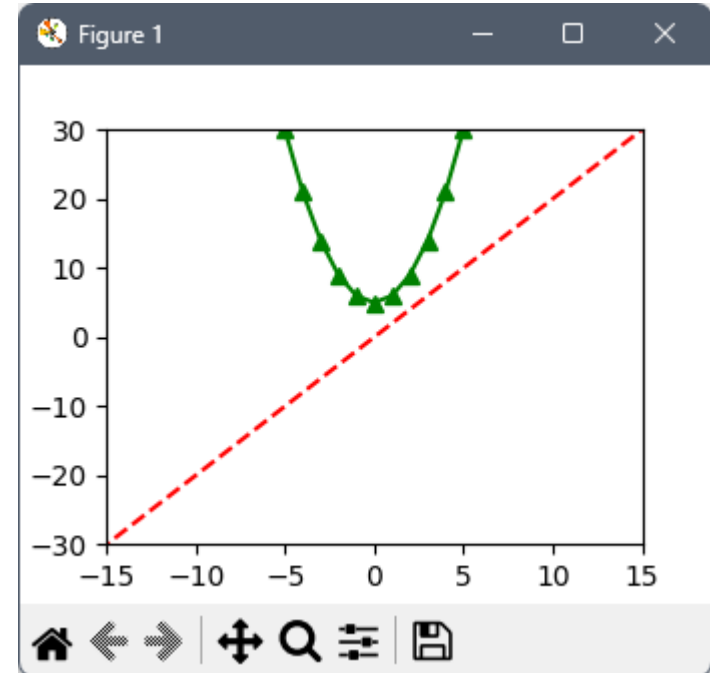
```
plt.plot([15.6, 14.2, 16.3, 18.2, 17.1, 20.2, 22.4], "sb")  
plt.show()
```



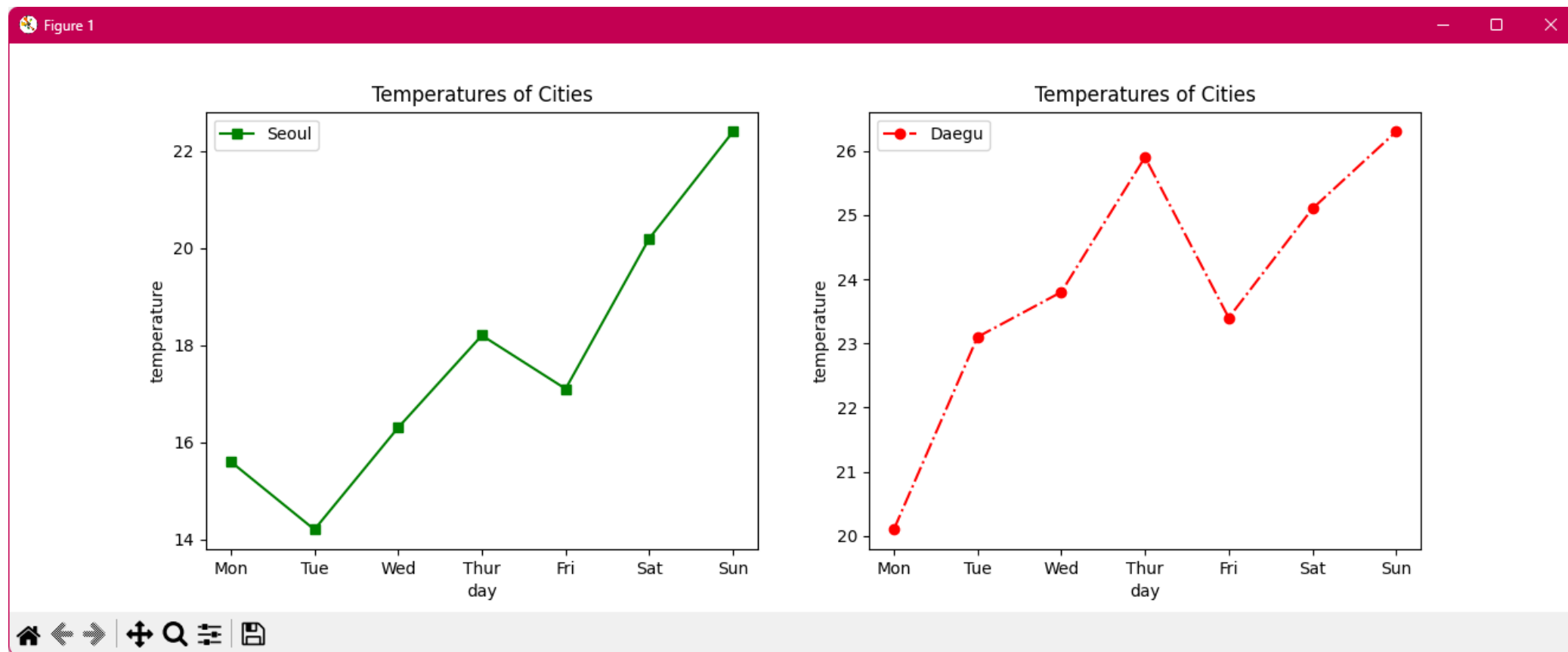
함수를 그래프로 그리기

- 두개의 함수 $y = 2x$, $y = x^2 + 5$ 를 그래프로 한번에 그리기

```
1 import matplotlib.pyplot as plt
2
3 r=[x for x in range(-15,15)]
4 y1=[2*x for x in r]
5 y2=[x**2+5 for x in r]
6 plt.plot(r,y1,'r--',r,y2,'g^-')
7 plt.axis([-15,15,-30,30])
8     #그림이 그려지는 범위 지정
9 plt.show()
```



한 창에 여러 개의 그래프 표시



```
1 import matplotlib.pyplot as plt
2 #plt.subplot(총 행 개수, 총 열 개수, 그래프가 표시될 면번호)
3 #면 번호는 1번부터 시작
4
5 plt.subplot(1, 2, 1)#1행 2열에서 1면에 그래프 표시
6 X = [ "Mon", "Tue", "Wed", "Thur", "Fri", "Sat", "Sun" ]
7 Y1 = [15.6, 14.2, 16.3, 18.2, 17.1, 20.2, 22.4]
8
9 plt.plot(X, Y1, label="Seoul",color='green', marker='s' , linestyle='solid')
10 plt.xlabel("day")
11 plt.ylabel("temperature")
12 plt.legend(loc="upper left")
13 plt.title("Temperatures of Cities")
14
15 plt.subplot(1,2,2)#1행 2열에서 2면에 그래프 표시
16 X = [ "Mon", "Tue", "Wed", "Thur", "Fri", "Sat", "Sun" ]
17 Y2 = [20.1, 23.1, 23.8, 25.9, 23.4, 25.1, 26.3]
18
19 plt.plot(X, Y2, label="Daegu", color='red',marker='o',linestyle='dashdot')
20 plt.xlabel("day")
21 plt.ylabel("temperature")
22 plt.legend(loc="upper left")
23 plt.title("Temperatures of Cities")
24
25 plt.show()
```


막대 그래프

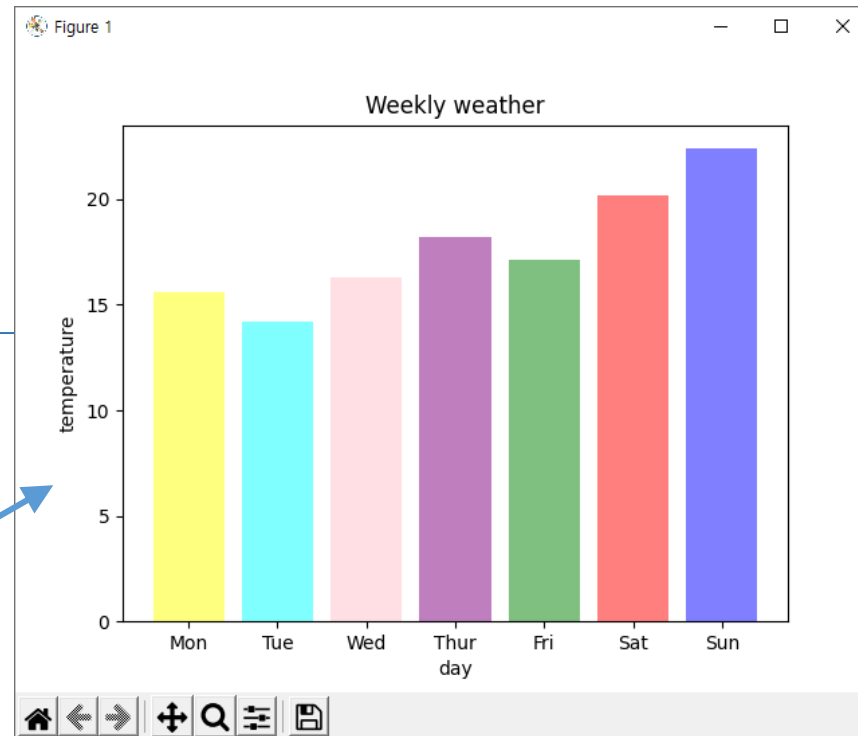
- bar() 메소드 사용 (옵션 alpha=0.5)

```
import matplotlib.pyplot as plt

X = [ "Mon", "Tue", "Wed", "Thur", "Fri", "Sat", "Sun" ]
Y = [ 15.6, 14.2, 16.3, 18.2, 17.1, 20.2, 22.4 ]
plt.bar(X, Y, color= c )
plt.title('Weekly weather')
plt.xlabel('day')
plt.ylabel('temperature')
plt.show()
```

color 매개변수의 값을 리스트 형태로 표시할 경우 막대마다 다른 색 지정 가능

c = ['yellow', 'cyan', 'pink', 'purple', 'green', 'red', 'blue']

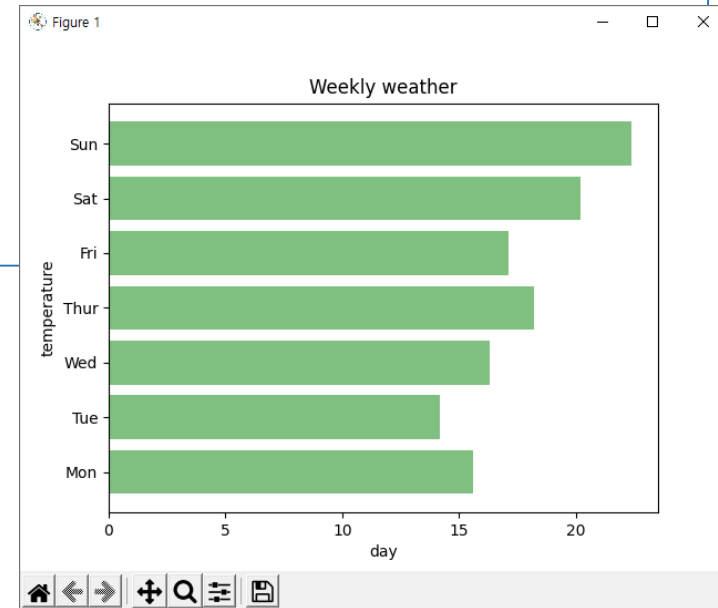


가로 막대 그래프

- barh() 메소드 사용

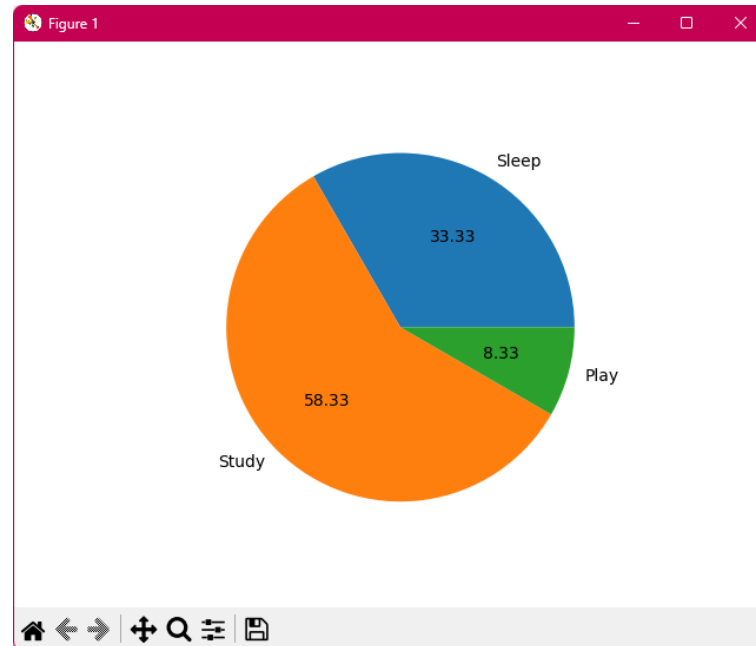
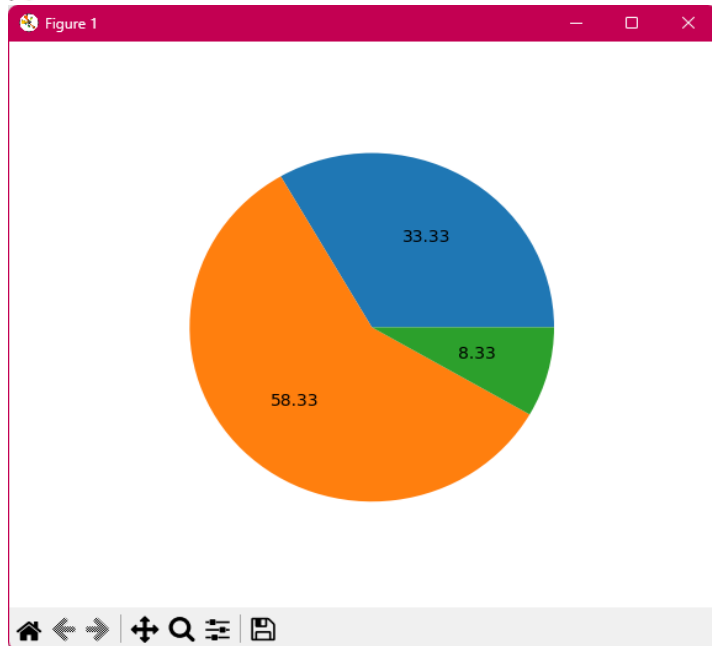
```
import matplotlib.pyplot as plt

X = [ "Mon", "Tue", "Wed", "Thur", "Fri", "Sat", "Sun" ]
Y = [15.6, 14.2, 16.3, 18.2, 17.1, 20.2, 22.4]
plt.barh(X, Y,color= 'g',alpha=0.5)
plt.title('Weekly weather')
plt.xlabel('day')
plt.ylabel('temperature')
plt.show()
```



파이형 그래프

```
1 import matplotlib.pyplot as plt
2 times = [8, 14, 2]
3 timelabels = ["Sleep", "Study", "Play"]
4
5 plt.pie(times, autopct = "%.2f")
6 #옵션 autopct는 표시할 소수점 이하 자리수
7 plt.show()
```

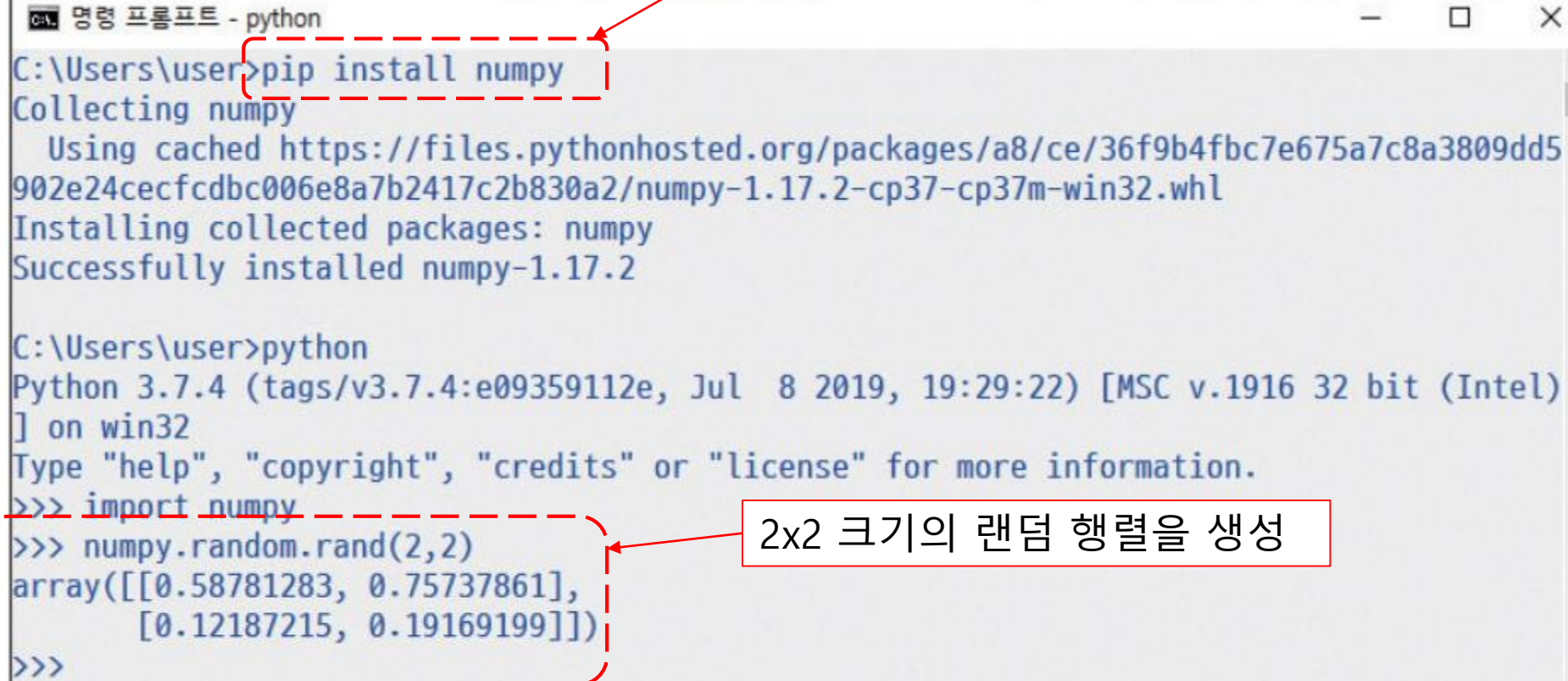


넘파이 라이브러리

- 넘파이|NumPy(Numerical Python)
 - 파이썬의 과학계산을 위한 가장 기본적인 라이브러리
 - 행렬, 벡터 연산을 위한 라이브러리로 빠른 처리속도가 장점 (C언어로 작성)
 - 다차원 배열과 행렬 객체가 포함
 - 처리 속도가 중요한 인공지능이나 데이터 과학에서는 파이썬의 리스트 대신에 넘파이를 선호한다
 - pip를 이용하여 명령행에서 numpy 설치

설치와 테스트

numpy 모듈 설치



```
C:\Users\user>pip install numpy
Collecting numpy
  Using cached https://files.pythonhosted.org/packages/a8/ce/36f9b4fbc7e675a7c8a3809dd5902e24cecfcdabc006e8a7b2417c2b830a2/numpy-1.17.2-cp37-cp37m-win32.whl
Installing collected packages: numpy
Successfully installed numpy-1.17.2

C:\Users\user>python
Python 3.7.4 (tags/v3.7.4:e09359112e, Jul  8 2019, 19:29:22) [MSC v.1916 32 bit (Intel)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import numpy
>>> numpy.random.rand(2,2)
array([[0.58781283, 0.75737861],
       [0.12187215, 0.19169199]])
>>>
```

The screenshot shows a Windows command prompt window titled "명령 프롬프트 - python". The first part shows the command `pip install numpy` being executed, with the output indicating that numpy-1.17.2 was successfully installed. The second part shows the command `python` being executed, which opens a Python 3.7.4 shell. In the shell, the commands `import numpy` and `numpy.random.rand(2,2)` are entered, resulting in a 2x2 array of random values. Red dashed boxes and arrows highlight the `pip install numpy` command and the `numpy.random.rand(2,2)` command, with labels explaining their purpose.

2x2 크기의 랜덤 행렬을 생성

[그림 12-1] pip를 이용한 numpy 설치화면과 간단한 테스트 코드

넘파이 라이브러리

대화창 실습 : numpy의 ndarray 사용

```
>>> import numpy as np
>>> a = np.array([1, 2, 3]) # 넘파이 1차원 배열 객체의 생성
>>> a
array([1, 2, 3])
>>> a.shape      # a 객체의 형태(shape)
(3,) #1차원 배열의 원소수가 3개임을 알려줌
>>> a.ndim       # a 객체의 차원
1
>>> a.dtype      # a 객체 내부 자료형
dtype('int32')
>>> a.itemsize   # a 객체 내부 자료형이 차지하는 메모리 크기(byte)
4
>>> a.size       # a 객체의 전체 크기(항목의 수)
3
```

ndarray

- 넘파이의 가장 핵심적인 객체, 다차원 배열을 처리하며 다음과 같은 속성들이 있다

```
a = np.array([1, 2, 3])
```

1	2	3
---	---	---

shape은 생성된 배열객체의 형태를 튜플 타입으로 반환함

```
a.shape : (3,)
a.ndim : 1
a.dtype : dtype('int32')
a.size : 3
a.itemsize : 4
```

[그림 12-2] 넘파이의 ndarray a와 그 속성들

shape, size 속성

```
import numpy as np
s=np.array([[1,2,3],[3,4,5]])
print('shape=',s.shape)
print('size=',s.size)
```

실행 결과

```
shape= (2, 3)
size= 6
```


배열 덧셈

- 넘파이 배열에는 $+$, $*$ 등의 수학적 연산자를 적용가능

대화창 실습 : 넘파이와 데이터 형

```
import numpy as np
```

```
a = np.array([1, 2, 3], dtype = 'int32')  
b = np.array([4, 5, 6], dtype = 'int64')
```

```
print(a.dtype)  
print(b.dtype)
```

```
c = a + b  
print(a, '+', b, '=', end='')  
print(c)
```

```
print(c.dtype)
```

1	2	3
---	---	---

+

4	5	6
---	---	---

=

5	7	9
---	---	---

실행 결과

```
int32  
int64  
[1 2 3] + [4 5 6] =[5 7 9]  
int64
```

BMI 지수 구하기

```
import numpy as np

heights = [ 1.83, 1.76, 1.69, 1.86, 1.77, 1.73 ]
weights = [ 86, 74, 59, 95, 80, 68 ]

np_h = np.array(heights)
np_w = np.array(weights)

bmi = np_w/(np_h**2)
print(bmi)
```

주의



주의 : ndarray 배열을 생성할 때 주의할 점

1. 넘파이의 배열 ndarray을 생성할 때, 반드시 대괄호를 사용하여 리스트 형식의 데이터를 만들어서 array() 함수의 인자로 넣어야 한다.

```
>>> a = np.array([1, 2, 3, 4])
```

만일 리스트 형식으로 하지 않고 쉼표로 구분해서 입력할 경우 다음과 같은 오류가 발생된다.

```
>>> a = np.array(1, 2, 3, 4) # 잘못된 입력
```

```
...
```

```
ValueError: only 2 non-keyword arguments accepted
```

주의

2. 넘파이의 ndarray는 리스트와는 달리, 서로 다른 자료형의 값을 원소로 가질 수 없다.

```
>>> a = np.array([1, 'two', 3, 4], dtype = np.int32)
...
ValueError: invalid literal for int() with base 10: 'two'
```

만일 다음과 같이 자료형을 명시하지 않을 경우 'two'라는 원소의 자료형인 str 형으로 자동 형 변환이 일어난다. 이 경우 모든 원소들은 문자열 형이되어 정수의 덧셈, 뺄셈 등의 연산을 사용할 수 없다.

```
>>> a = np.array([1, 'two', 3, 4])
>>> a
array(['1', 'two', '3', '4'], dtype = '<U21')
```

배열의 데이터 타입 지정 방식



NOTE : 넘파이 배열의 데이터 타입을 지정하는 두 가지 방법

넘파이 배열의 데이터 타입을 지정하는 방법에는 두 가지가 있다.

1. `dtype = np.int32` 와 같이 `np`의 `int32` 속성 값으로 지정하기

```
>>> a = np.array([1, 2, 3, 4], dtype = np.int32)
```

2. `dtype = 'int32'` 와 같이 문자열 형식으로 속성 값 지정하기

```
>>> a = np.array([1, 2, 3, 4], dtype = 'int32')
```

- numpy의 ndarray의 속성

속성	설명
ndim	배열 축 혹은 차원의 갯수.
shape	배열의 차원으로 (m, n) 형식의 튜플 형이다. 이 때, m과 n은 각 차원의 원소의 크기를 알려주는 정수 값이다.
size	배열의 원소의 갯수이다. 이 갯수는 shape내의 원소의 크기의 곱과 같다. 즉 (m, n) shape 배열의 size는 $m*n$ 이다.
dtype	배열내의 원소의 형을 기술하는 객체이다. numpy는 파이썬 표준 형을 사용할 수 있으나 numpy 자체의 자료형인 bool_, character, int_, int8, int16, int32, int64, float, float8, float16_, float32, float64, complex_, complex64, object_ 형을 사용할 수 있다.
itemsize	배열내의 원소의 크기를 바이트 단위 로 기술한다. 예를 들어 int32 자료형의 크기는 $32/8=4$ 바이트가 된다.
data	배열의 실제 원소를 포함하고 있는 버퍼.



LAB 11-1 : ndarray 객체 생성하기 그리고 속성 알아보기

1. 0에서 9까지의 정수 값을 가지는 ndarray 객체 a를 넘파이를 이용하여 작성하여 다음과 같이 출력하여야.

```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

2. range() 함수를 사용하여 0에서 9까지의 정수 값을 가지는 ndarray 객체 b를 만들고 문제 1의 결과와 같이 나타나도록 하여라.

3. 문제 2의 코드를 수정하여 0에서 9까지의 정수 값 중에서 다음과 같이 짝수를 가지는 ndarray 객체 c를 출력하여야.

```
array([0, 2, 4, 6, 8])
```

4. 문제 3번 ndarray 객체 c의 shape, ndim, dtype, size, itemsize를 다음과 같이 출력하여야.

```
c.shape = (5,)
c.ndim = 1
c.dtype = int64
c.size = 5
c.itemsize = 8
```

```
import numpy as np
a=np.array([0,1,2,3,4,5,6,7,8,9])
print('a=',a)
b=np.array(range(10))
print('b=',b)
c=np.arange(0,10,2)
print('c=',c)
```

배열 메소드와 주요 함수

max(), min(), mean()

대화창 실습 : ndarray의 메소드

```
>>> a = np.array([1, 2, 3])  
# 1차원 ndarray 배열 생성  
>>> a.max()      # 가장 큰 값을 반환  
3  
>>> a.min()      # 가장 작은 값을 반환  
1  
>>> a.mean()     # 평균 값을 반환  
2.0
```

flatten() #2차원 → 1차원

대화창 실습 : ndarray의 flatten() 메소드

```
>>> a = np.array([[1, 1], [2, 2], [3, 3]])  
>>> a.flatten()  
# ndarray 배열의 평탄화 메소드  
array([1, 1, 2, 2, 3, 3])
```


ndarray의 메소드와 주요 함수

append() 함수

대화창 실습 : ndarray의 append() 함수

```
>>> a = np.array([1, 2, 3])
>>> b = np.array([[4, 5, 6], [7, 8, 9]])
>>> np.append(a, b)
array([1, 2, 3, 4, 5, 6, 7, 8, 9])
>>> np.append([a], b, axis = 0)
# [a]를 통해 2차원 배열로 만들어야 함
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
```

rand() 함수

대화창 실습 : ndarray의 rand() 함수

```
>>> np.random.rand(3, 3)
# (3, 3) shape의 난수 생성
array([[0.63208882, 0.72259476, 0.15125742],
       [0.60731818, 0.20682056, 0.51958311],
       [0.644331  , 0.91940484, 0.83990604]])
```

ndarray의 메소드와 주요 함수

- randint() 함수

Ndarray의 randint()함수

```
>>> np.random.randint(0, 10, size = 10)
      # 0에서 10까지의 10개의 난수 생성
array([2, 0, 8, 2, 7, 0, 1, 3, 1, 3])
```



LAB 12-2 : ndarray 객체의 메소드와 함수

1. [23, 45, 67, 7, 2, 30, 34, 82]의 정수 값을 가지는 ndarray 객체 a를 생성하여 다음과 같이 출력하여야.

```
a = array([23 45 67 7 2 30 34 82])
```

이 ndarray의 max(), min(), mean() 메소드를 이용하여 최댓값, 최솟값, 평균을 다음과 같이 출력하여야.

```
최댓값 : 82
최솟값 : 2
평균 : 36.25
```

2. numpy.random.randint() 함수를 사용하여 0에서 99까지의 정수 값 10개를 랜덤하게 생성하십시오. 이 10개의 값을 b라는 ndarray에 넣고 ndarray에서 최댓값, 최솟값, 평균을 다음과 같이 출력하여야(b 값은 랜덤하게 생성되므로 다음 화면의 값과 일치하지 않음).

```
b = [25 2 9 86 93 73 15 53 67 20]
최댓값 : 93
최솟값 : 2
평균 : 44.3
```

3. 위의 문제 1의 a 배열과 문제 2의 b 배열을 append() 하여 다음과 같은 배열 c를 생성하여 출력하십시오.

```
c = [23 45 67 7 2 30 34 82, 25 2 9 86 93 73 15 53 67 20]
```

LAB 12-2, 1번 코드

```
import numpy as np
a = np.array([23, 45, 67, 7, 2, 30, 34, 82])
print('a = array({})'.format(a))

print('최댓값 : {}'.format(a.max()))
print('최솟값 : {}'.format(a.min()))
print('평균 : {}'.format(a.mean()))
```

LAB 12-2, 2번 코드

```
import numpy as np
b = np.random.randint(0, 100, size=10)
print('b = {}'.format(b))

print('최댓값 : {}'.format(b.max()))
print('최솟값 : {}'.format(b.min()))
print('평균 : {}'.format(b.mean()))
```

LAB 12-2, 3번 코드

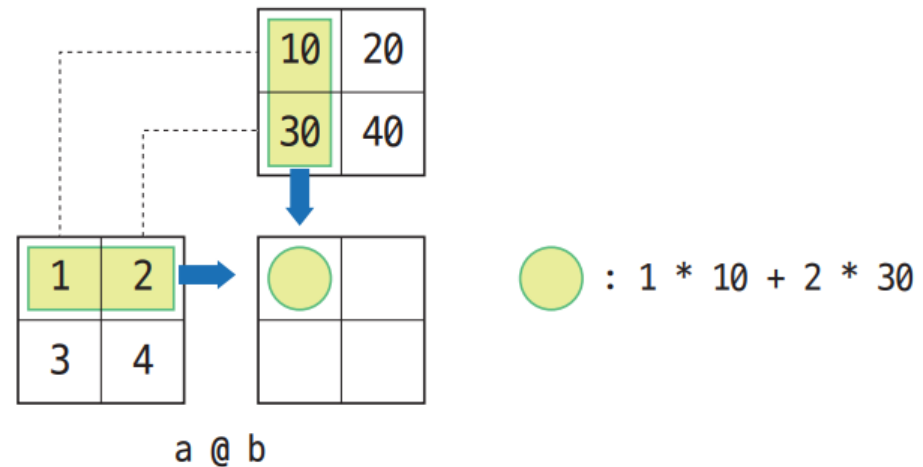
```
import numpy as np
a = np.array([23, 45, 67, 7, 2, 30, 34, 82])
b = np.random.randint(0, 100, size=10)
c = np.append(a, b)
print('c = {}'.format(c))
```

행렬 곱 matmul(),@

대화창 실습 : 행렬 곱 함수 matmul() 실습

```
>>> np.matmul(a, b)
array([[ 70, 100],
       [150, 220]])
>>> a @ b
array([[ 70, 100],
       [150, 220]])
>>> a[0,0] * b[0,0] + a[0,1] * b[1,0]
70
>>> a[0,0] * b[0,1] + a[0,1] * b[1,1]
100
>>> a[1,0] * b[0,0] + a[1,1] * b[1,0]
150
>>> a[1,0] * b[0,1] + a[1,1] * b[1,1]
220
```

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} @ \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} = \begin{bmatrix} 1*10+2*30 & 1*20+2*40 \\ 3*10+4*30 & 3*20+4*40 \end{bmatrix}$$



[그림 12-5] 넘파이의 a @ b 연산

대화창 실습 : 2차원 ndarray의 덧셈, 뺄셈, 곱셈, 나눗셈, 제곱 연산

```
>>> a = np.array([[1, 2], [3, 4]])
>>> a + 1          # 행렬의 각 성분에 대한 덧셈
array([[2, 3],
       [4, 5]])
>>> a - 1          # 행렬의 각 성분에 대한 뺄셈
array([[0, 1],
       [2, 3]])
>>> a * 100        # 행렬의 각 성분에 대한 곱셈
array([[100, 200],
       [300, 400]])
>>> a / 100        # 행렬의 각 성분에 대한 나눗셈
array([[0.01, 0.02],
       [0.03, 0.04]])
>>> a ** 2         # 행렬의 각 성분에 대한 제곱연산
array([[ 1,  4],
       [ 9, 16]])
```

1	2
3	4

+ 1 =

1	2
3	4

* 100 =

1	2
3	4

** 2 =



LAB 12-3 : 다차원 행렬의 생성과 활용

1. 1에서 9까지의 모든 정수 값을 크기 순서대로 가지는 3x3 크기의 행렬 a를 생성하고, 모든 성분의 값이 2인 3x3 크기의 행렬 b를 생성하여라.

```
a = [[1 2 3]
      [4 5 6]
      [7 8 9]]
```

```
b = [[2 2 2]
      [2 2 2]
      [2 2 2]]
```

2. 다음 행렬 연산의 결과를 예상한 후 실행하고 그 결과를 적으시오.

- 1) $a + b$
- 2) $a - b$
- 3) $a * b$
- 4) a / b
- 5) $a @ b$
- 6) $a ** 2$

ndarray의 생성

모든 값이 0인 2x3 크기 행렬

대화창 실습 : 초기값을 가지는 행렬의 생성

```
>>> np.zeros((2, 3))
```

```
array([[0., 0., 0.],  
       [0., 0., 0.]])
```

모든 값이 1인 2x3 크기 행렬

```
>>> np.ones((2, 3))
```

```
array([[1., 1., 1.],  
       [1., 1., 1.]])
```

모든 값이 100인 2x3 행렬

```
>>> np.full((2, 3), 100)
```

```
array([[100, 100, 100],  
       [100, 100, 100]])
```

```
>>>... 이어서...
```

대화창 실습 : 초기값을 가지는 행렬의 생성

3x3 크기의 단위행렬

```
>>> np.eye(3)
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])
```

2x3 크기의 랜덤행렬

```
>>> np.random.random((2, 3))
array([[0.87143684, 0.44538612, 0.11908973],
       [0.87548557, 0.57502347, 0.87641631]])
```

- `eye(n)` 함수를 이용하면 $n \times n$ 크기의 단위행렬(identity matrix)을 생성할 수 있다.
- 정방행렬 중에서 대각선 성분이 1이고 나머지 성분이 0인 행렬

arange(0, 10)

0	1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

[그림 12-6] arange(0,10) 실행 결과

arange(0, 10, 2)

0	1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

step=2

[그림 12-7] arange(0, 10, 2) 실행과 스텝 값

0	2	4	6	8
---	---	---	---	---

[그림 12-8] arange(0, 10, 2) 실행 결과

```
>>> list(range(10))  
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]  
>>> list(range(0, 1.0, 0.3))
```

Traceback (most recent call last):

File "<pyshell#2>", line 1, in <module>

list(range(0, 1.0, 0.3))

TypeError: 'float' object cannot be interpreted as an integer

파이썬 리스트는 실수 step값을 사용할 수 없음

대화창 실습 : arange() 함수를 이용한 배열 생성

```
>>> import numpy as np
```

```
>>> np.arange(0, 10)
```

```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
>>> np.arange(0, 10, 2)
```

```
array([0, 2, 4, 6, 8])
```

```
>>> np.arange(0, 10, 3)
```

```
array([0, 3, 6, 9])
```

실수 인수 사용 가능

```
>>> np.arange(0.0, 1.0, 0.2)
```

```
array([0. , 0.2, 0.4, 0.6, 0.8])
```

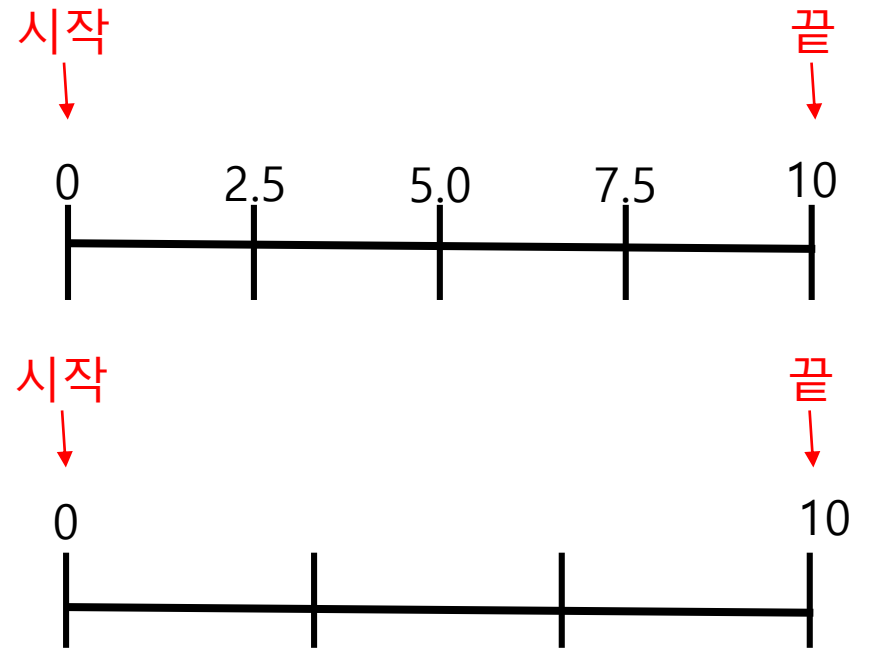
linspace() 메소드

- linspace() : 동일한 간격을 가진 연속된 값을 생성
- linspace(시작, 끝, 생성할 수의 개수)
- 디폴트 간격의 수는 50개

```
1 import numpy as np
2 x=np.linspace(0, 10,5)
3 print(x)
4 x=np.linspace(0, 10)
5 # 디폴트 개수가 50개
6 print(x)
7 y=np.linspace(0, 10, 4)
8 print(y)
```

실행 결과

```
[ 0.  2.5  5.  7.5 10.]
[ 0.          3.33333333  6.66666667 10.          ]
```



logspace() 메소드

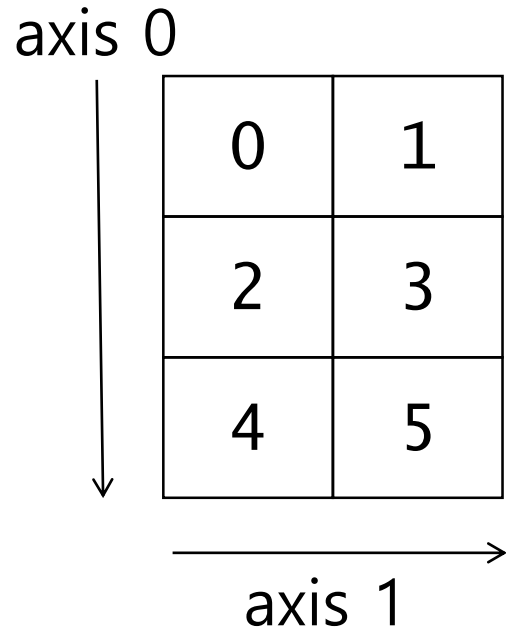
- 로그 스케일로 수 생성
- `logspace(start, stop, n)`
→ 10^{start} 에서 10^{stop} 까지 n 개 생성

```
x=np.logspace(0, 100 ,5)  
print(x)
```

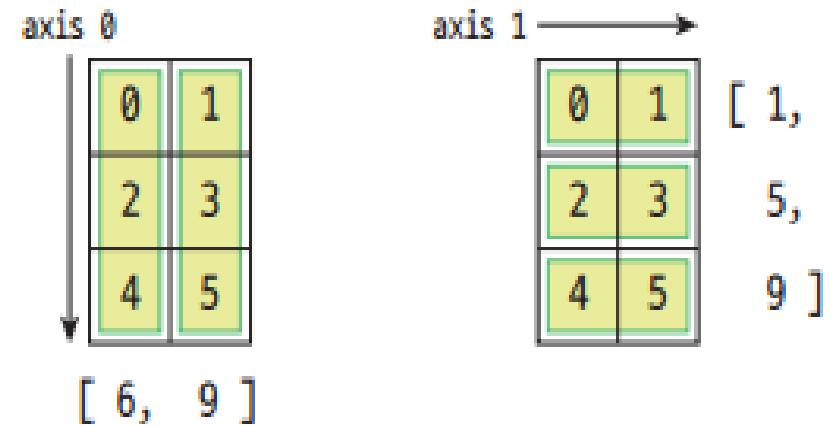
실행 결과

```
[1.e+000 1.e+025 1.e+050 1.e+075 1.e+100]
```

다차원 배열의 축



[그림 12-10] 2차원 배열과 축 1과 축 2의 방향



[그림 12-11] axis 0과 axis 1에 대하여 각각 `sum()` 함수를 수행한 결과

sum() 메소드

대화창 실습 : sum() 함수와 axis에 따른 원소의 합

```
>>> import numpy as np
>>> a = np.arange(0, 6).reshape(3, 2)
>>> a
array([[0, 1],
       [2, 3],
       [4, 5]])
>>> a.sum()      # 행렬의 모든 원소의 합
15
>>> a.sum(axis = 0)
# 0축 방향(행 방향) 원소의 합
array([6, 9])
```

대화창 실습 : 1축 방향(열 방향) 원소의 합

```
>>> import numpy as np
>>> a.sum(axis = 1)
#1축 방향(열방향) 원소의 합
array([1, 5, 9])
```

min(), max() 메소드

대화창 실습 : 0축과 1축 방향 원소의 최솟값, 최댓값

```
>>> import numpy as np
>>> a.min(axis = 0)    # 0축 방향 원소의 최솟값
array([0, 1])
>>> a.min(axis = 1)    # 1축 방향 원소의 최솟값
array([0, 2, 4])
>>> a.max(axis = 0)    # 0축 방향 원소의 최댓값
array([4, 5])
>>> a.max(axis = 1)    # 1축 방향 원소의 최댓값
array([1, 3, 5])
```

insert() 메소드

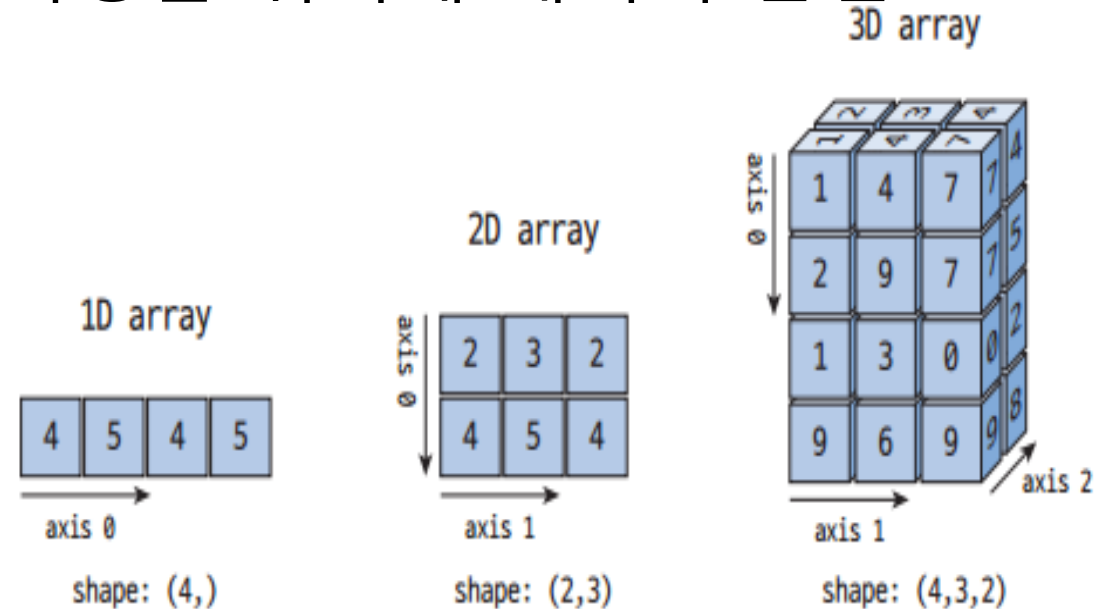
- 삽입될 위치(index)를 지정하여 지정한 위치에 데이터 삽입

대화창 실습 : ndarray의 insert() 함수

```
>>> a = np.array([1, 3, 4])
>>> np.insert(a, 1, 2)
array([1, 2, 3, 4])

>>> a = np.array([[1, 1], [2, 2], [3, 3]])
>>> np.insert(a, 1, 4, axis = 0)
array([[1, 1],
       [4, 4],
       [2, 2],
       [3, 3]])

>>> np.insert(a, 1, 4, axis = 1)
array([[1, 4, 1],
       [2, 4, 2],
       [3, 4, 3]])
```



[그림 12-12] 1차원, 2차원, 3차원 배열의 형태와 각 배열의 축 방향

선형 방정식 풀이, 행렬식

$$2x + 3y = 1$$

$$x - 2y = 4$$

[수식 12-1] 선형 연립방정식

코드 12-1 : 선형 연립 방정식 풀이

numpy_linear_ex.py

```
import numpy as np

a = np.array([[2, 3], [1, -2]])
b = np.array([1, 4])
x = np.linalg.solve(a, b)
print(x)
```

실행결과

[2. -1.]

• 2×2행렬의 행렬식 $\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc$

• 3×3행렬의 행렬식 $\det \begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix} = aei + bgf + cdh - ceg - bdi - afh$

[수식 12-2] 2×2 행렬의 행렬식과 3×3 행렬의 행렬식

대화창 실습 : 행렬식과 det() 함수

```
>>> a = np.array([[1, 2], [3, 4]])
```

```
>>> np.linalg.det(a)
```

```
-2.0000000000000004
```

```
>>> b = np.array([[1, 2], [3, -6]])
```

```
>>> np.linalg.det(b)
```

```
-12.0
```

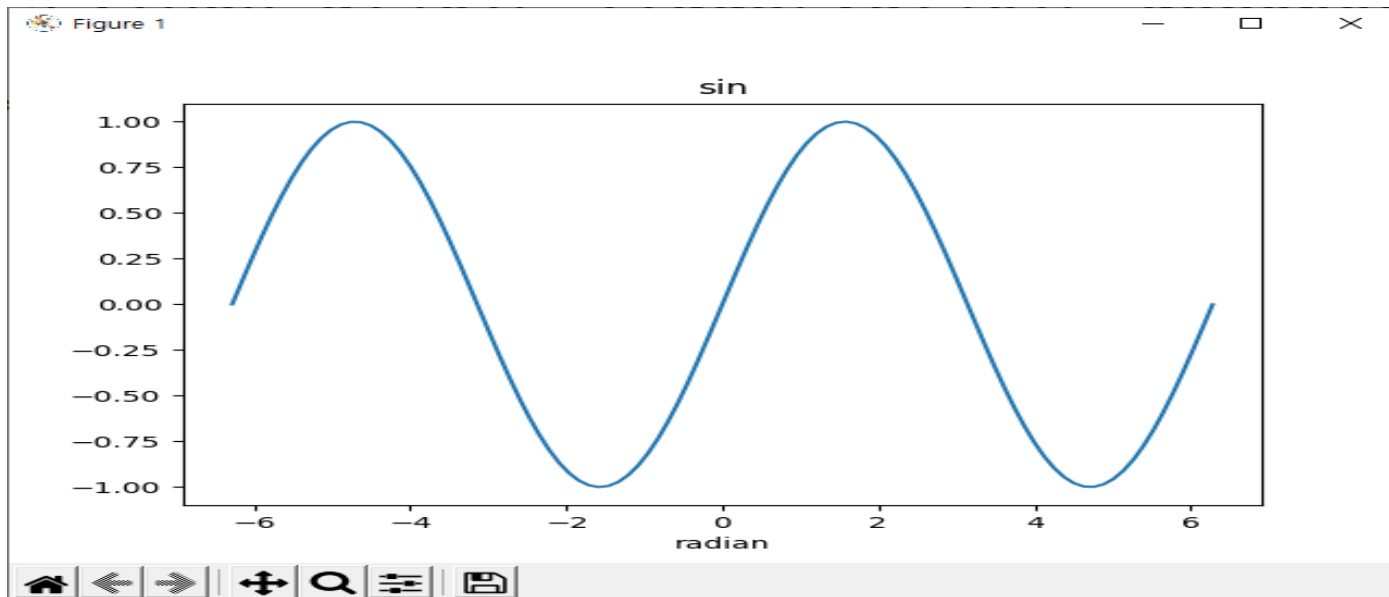
```
>>> b = np.array([[1, 2], [1, 2]])
```

```
>>> np.linalg.det(b)
```

```
0.0
```

싸인 함수 그리기

- linspace() 함수를 사용하여 일정 간격의 데이터를 만들고 넘파이의 sin() 함수에 이 데이터를 전달하여서 싸인값을 얻고, 싸인 그래프 그린다



```
import matplotlib.pyplot as plt
import numpy as np
```

-2π 에서 $+2\pi$ 까지 100개의 데이터를 균일하게 생성

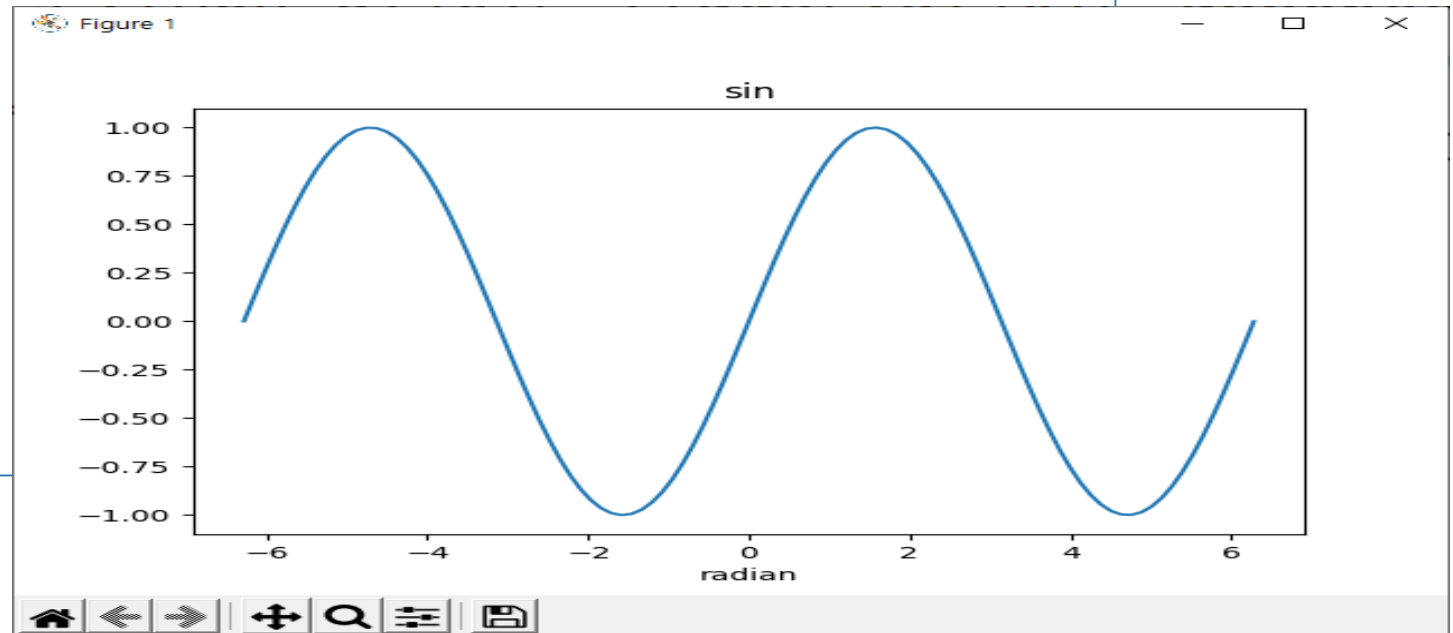
```
X = np.linspace(-2 * np.pi, 2 * np.pi, 100)
```

넘파이 배열에 $\sin()$ 함수를 적용

```
Y1 = np.sin(X)
```

```
#Y2 = 3 * np.sin(X)
```

```
plt.plot(X, Y1)
plt.title('sin')
plt.xlabel('radian')
plt.show()
```





Questions?